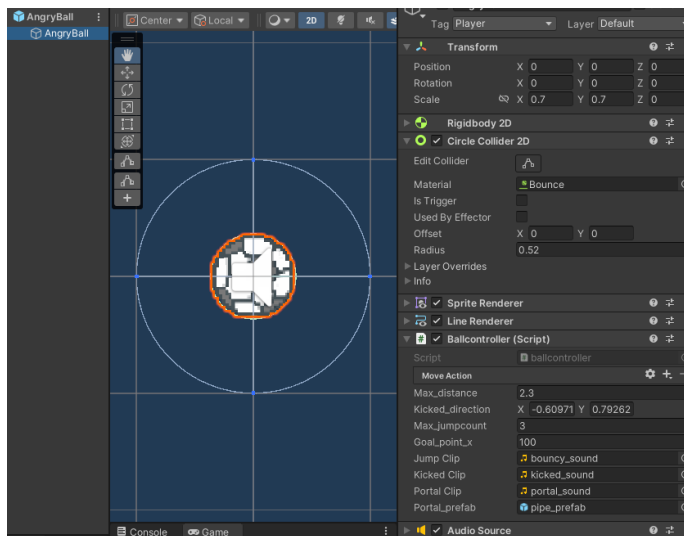


1. 드래그 거리 기반 조작 및 기본 움직임 시스템 (ballcontroller.cs)



Max_distance: 점프 최대 거리

Kicked_direction: 장애물 닿았을 때 날아갈 방향

Max_jumpcount: 최대 점프 횟수

Goal_point_x: 진행도 UI 사용될 값

- 드래그 거리 기반 발사 및 제한 시스템

화면상 거리를 라인렌더러 거리의 차이가 있어서 드래그 거리를 픽셀 상 거리로 바꿔줘야 하기 때문에 드래그 해보며 찾은 적정값인 65를 곱해줌. 후에 팀원들의 난이도 피드백 이후 브레이크 기능 추가. 브레이크 시 점프 횟수 0으로 페널티

```
if (Input.GetMouseButtonDown(0))
{
    start_point = Input.mousePosition;
    lineRenderer.enabled = true;
    on_calculate = true;
}
else if (Input.GetMouseButton(0))
{
    if (!on_calculate) return;
    Vector2 current = Input.mousePosition;
    Vector2 screenVec = current - start_point;
    float screenDist = (current_max_distance * 65 < screenVec.magnitude ? current_max_distance * 65 : screenVec.magnitude);
    Vector2 screenDir = screenVec.normalized;

    Vector3 worldDir = (Vector3)(screenDir * screenDist * 0.015f);
    Vector3 worldStart = transform.position;
    Vector3 worldEnd = worldStart + worldDir;

    lineRenderer.SetPosition(0, worldStart);
    lineRenderer.SetPosition(1, worldEnd);
}
```

```
else if (Input.GetMouseButtonUp(0))
{
    if (!on_calculate) return;
    end_point = Input.mousePosition;
    lineRenderer.enabled = false;

    CurrentJumpCount -= 1;

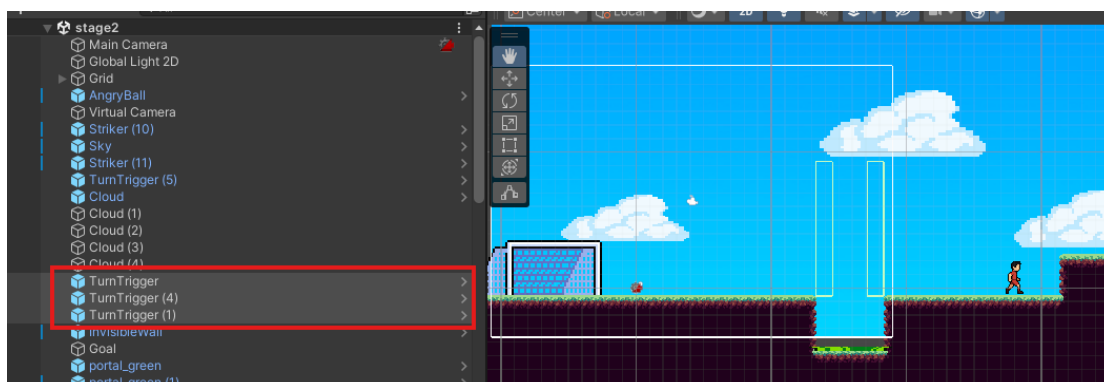
    direction = (start_point - end_point).normalized;
    distance = Vector2.Distance(start_point, end_point) / 65;
    distance = Mathf.Min(distance, current_max_distance);

    PlaySound(jumpClip);
    rigidbody2d.AddForce(direction * distance * 8f, ForceMode2D.Impulse);
    on_calculate = false;

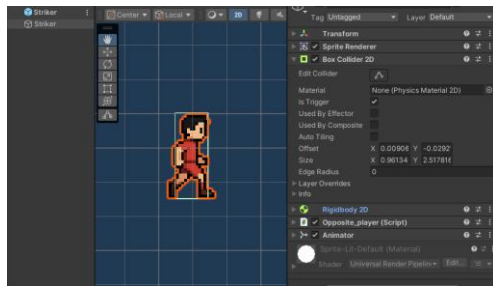
    start_point = Vector2.zero;
    end_point = Vector2.zero;

    if (CurrentJumpCount < max_jumpcount)
    {
        current_max_distance = 0.85f * current_max_distance;
    }
}
```

2. 선수 오브젝트 구현 (istrigger 활성화한 상태로 통과 가능하도록 구현. 타일을 통과하여 떨어지지 않도록 kinematic 설정. turntrigger이라는 콜라이더만 있는 오브젝트를 만들어 구멍있는 지점들에 배치하여 움직이는 선수가 부딪혔을 때 돌도록 하여 땅 위 경로 안에서만 다니도록 함.)



- Striker



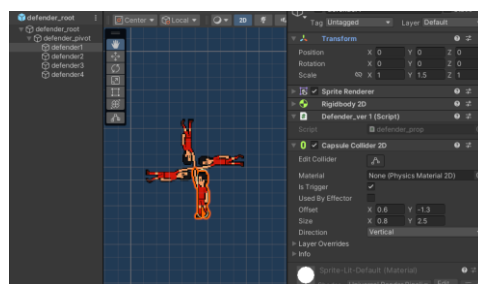
- tackleman



- standman

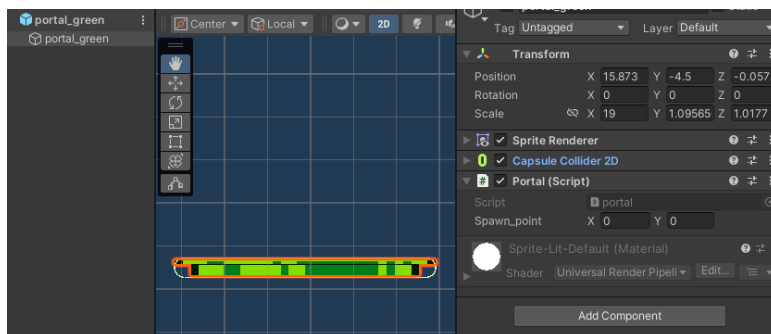


-defender_root



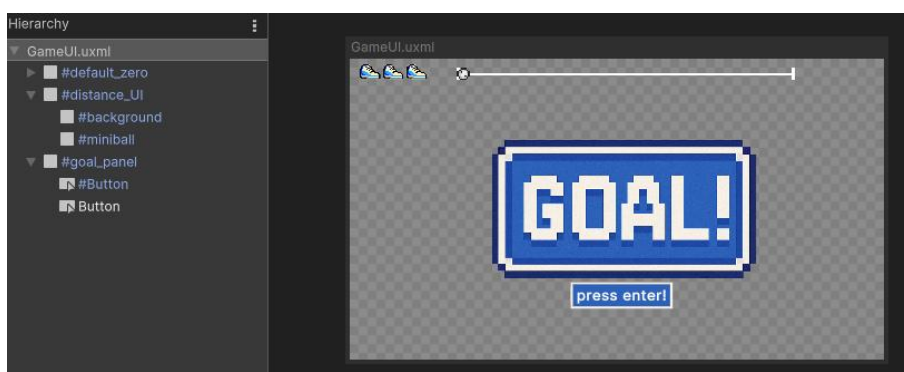
3. 바닥 포탈 장애물

- portal_green(구멍 속 있는 포탈, 스폰지점 커스텀 가능: spawn_point)

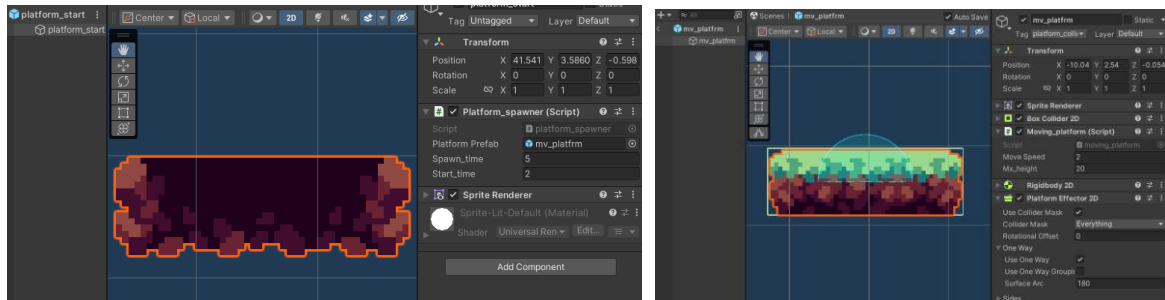


- 스폰 시 포탈이 바닥에서 나오며 그 다음 공이 나오도록 구현
(ballcontroller.cs/SpawnPortalAtPosition)

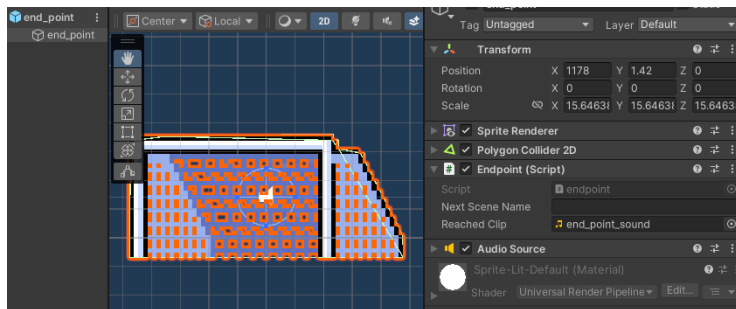
4. UI 기능 (진행도, 점프 잔여 횟수, 골대 진입 시 표시문. UI_handler.cs/ballcontroller.cs)



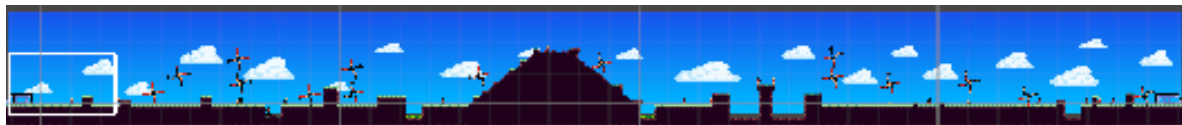
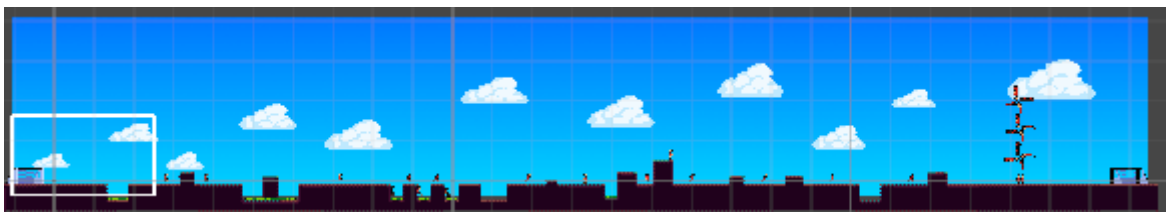
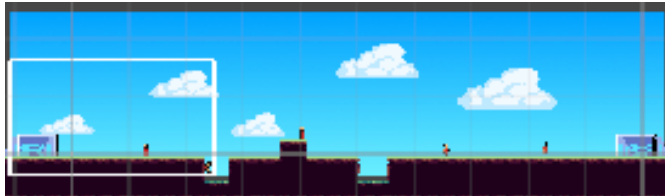
5. 움직이는 발판 (스포너: 스폰시간 및 주기 커스텀 가능 / 플랫폼 오브젝트)



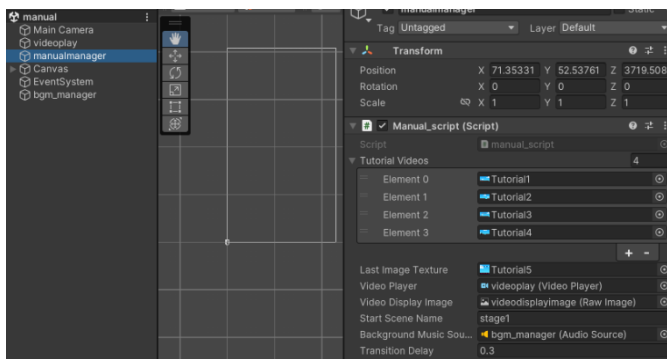
6. 골대 오브젝트 구현 (Next Scene Name: 해당 골대에 도달 후 이동 씬 이름)



7. 맵 장애물 구성("stage1", "stage2", "stage3") / 배경화면인 Sky로 카메라 제한범위 설정



8. 매뉴얼 및 엔딩 비디오 삽입, endscreen 씬 버튼 UI (end.cs)



9. 웹빌드 버전

https://github.com/ndog-bcat/dragandshoot_20251108 (본 프로젝트 레포지터리)

https://github.com/ndog-bcat/DAS_videos (비디오 URL 생성 목적 레포지터리)

https://github.com/ndog-bcat/drag_and_shoot_demo (비디오 URL 사용한 웹버전 빌드 레포지터리)